

Politechnika Gdańska
Wydział Elektroniki, Telekomunikacji i Informatyki

SPRAWOZDANIE Z PROJEKTU

**Temat: Natural-Language Weather Data Explorer
na platformie chmurowej AWS**

Autorzy:

Marek Marcinko (nr indeksu: 197870)
Szymon Kowalski (nr indeksu: 198055)

Gdańsk, Czerwiec 2026

Spis treści

1	Wstęp i cel projektu	2
2	Architektura systemu i zastosowane usługi	2
3	Szczegóły implementacji	2
3.1	Pobieranie danych (Ingestion Pipeline)	2
3.2	Przetwarzanie ETL w AWS Glue (Apache Spark)	3
3.3	Konfiguracja Warstwy Analitycznej w Amazon Athena	3
3.4	Asystent Q&A (Template-based Query Mapping)	3
3.5	Interfejs Webowy i API Gateway	4
4	Testowanie systemu i uzyskane wyniki	4
5	Podsumowanie i wnioski	4

1 Wstęp i cel projektu

Celem niniejszego projektu było zaprojektowanie oraz implementacja kompletnego potoku przetwarzania danych (data pipeline) typu end-to-end w chmurze Amazon Web Services (AWS). Głównym zadaniem systemu jest cykliczne pobieranie surowych danych pogodowych w formacie JSON z zewnętrznego serwisu OpenWeatherMap API, ich automatyczna walidacja, transformacja strukturalna przy użyciu silnika Apache Spark, a następnie udostępnienie interfejsu odpytywania danych w języku naturalnym (Natural Language Interface).

Projekt odpowiada na rzeczywiste wyzwania inżynierii danych związane z integracją danych w czasie rzeczywistym, optymalizacją kosztów przechowywania i przeszukiwania dużych wolumenów danych (columnar storage) oraz budową bezserwerowych (serverless) interfejsów analitycznych.

2 Architektura systemu i zastosowane usługi

System został zaprojektowany w oparciu o model serverless, co minimalizuje koszty utrzymania infrastruktury i zapewnia automatyczną skalowalność. Architektura składa się z pięciu głównych warstw:

1. **Warstwa Ingestii (Pobierania):** Usługa **AWS Lambda** uruchamiana cyklicznie, wykonująca zapytania HTTP do OpenWeatherMap REST API.
2. **Warstwa Składowania Danych Surowych (Raw Data Area):** Bucket **Amazon S3** ('weather-raw-data-*') przechowujący niezmienione pliki źródłowe JSON.
3. **Warstwa Przetwarzania i Walidacji (ETL):** Usługa **AWS Glue** uruchamiająca rozproszony skrypt **Apache Spark** w celu czyszczenia i transformacji danych.
4. **Warstwa Składowania Danych Przetworzonych (Curated Area):** Drugi bucket **Amazon S3** ('weather-curated-data-*') przechowujący dane w zoptymalizowanym formacie zorientowanym kolumnowo (**Parquet**).
5. **Warstwa Analityczna i Interfejs Użytkownika:** Usługa **Amazon Athena** jako bezserwerowy silnik zapytań SQL, funkcja **AWS Lambda** mapująca język naturalny na SQL oraz **Amazon API Gateway** wystawiający interfejs REST dla aplikacji klienckiej (HTML/JS).

3 Szczegóły implementacji

3.1 Pobieranie danych (Ingestion Pipeline)

Pierwsza funkcja Lambda pobiera dane dla określonej lokalizacji geograficznej (Gdańsk) przy użyciu klucza uwierzytelniającego API. Pobrana odpowiedź JSON jest natychmiast zapisywana w buckecie S3 z unikalnym znacznikiem czasu w nazwie pliku (np. 'weather_Gdansk_20260625122758.json'), co pozwala na zachowanie pełnej historii pomiarów.

3.2 Przetwarzanie ETL w AWS Glue (Apache Spark)

Do transformacji danych surowych do postaci ustrukturyzowanej wykorzystano zadanie Spark w usłudze AWS Glue. Skrypt Spark odczytuje katalog wejściowy S3, dokonuje rzutowania typów, spłaszcza strukturę zagnieżdżonego dokumentu JSON oraz filtruje wyłącznie kluczowe metryki biznesowe: temperaturę, wilgotność oraz ogólne warunki atmosferyczne. Wynik zapisywany jest w formacie Parquet z trybem dopisywania ('append').

Fragment kluczowej logiki transformacji przedstawia poniższy listing:

```
1 # Odczyt danych JSON
2 df = spark.read.json("s3://weather-raw-data-296066093533-us-east-1-an/")
3
4 # Transformacja i czyszczenie schematu
5 df_cleaned = df.selectExpr(
6     "name as city",
7     "main.temp as temp_kelvin",
8     "main.humidity as humidity",
9     "weather[0].main as weather_condition"
10 )
11
12 # Zapis do Curated Dataset w formacie Parquet
13 df_cleaned.write.mode("append").parquet("s3://weather-curated-data-296066093533-us-east-1-an/")
```

Listing 1: Fragment skryptu ETL w Apache Spark

3.3 Konfiguracja Warstwy Analitycznej w Amazon Athena

W usłudze Amazon Athena utworzono dedykowaną bazę danych 'weather_db' oraz zdefiniowano strukturę tabeli zewnętrznej 'curated_weather'. Tabela ta wskazuje bezpośrednio na fizyczną lokalizację plików Parquet w drugim bukiecie S3. Dzięki temu zapytania SQL są wykonywane bezpośrednio na plikach, bez konieczności ładowania danych do tradycyjnej bazy relacyjnej.

```
1 CREATE EXTERNAL TABLE IF NOT EXISTS weather_db.curated_weather (
2     city string,
3     temp_kelvin double,
4     humidity bigint,
5     weather_condition string
6 )
7 STORED AS PARQUET
8 LOCATION 's3://weather-curated-data-296066093533-us-east-1-an/';
```

Listing 2: Definicja tabeli zewnętrznej w Athena

3.4 Asystent Q&A (Template-based Query Mapping)

W celu realizacji dostępu za pomocą języka naturalnego, zaimplementowano funkcję Lambda 'WeatherAssistant'. Wykorzystuje ona technikę mapowania szablonów pytań na konkretne, bezpieczne zapytania SQL. Funkcja analizuje tekst przesłany przez użytkownika, generuje zapytanie, wywołuje asynchronicznie silnik Athena za pomocą biblioteki 'boto3', cyklicznie sprawdza status wykonania, a po zakończeniu przetwarza surowe wyniki (np. konwertuje temperaturę z Kelwinów na stopnie Celsjusza) i zwraca ustrukturyzowany dokument JSON.

3.5 Interfejs Webowy i API Gateway

Funkcja asystenta została zintegrowana z usługą Amazon API Gateway. Skonfigurowano endpoint typu REST z metodą 'POST' oraz włączono integrację typu Lambda Proxy. Ze względu na fakt, że klient jest uruchamiany lokalnie w przeglądarce, kluczowe było poprawne włączenie mechanizmu CORS (Cross-Origin Resource Sharing) zarówno na poziomie API Gateway, jak i poprzez dodanie nagłówka 'Access-Control-Allow-Origin' bezpośrednio w kodzie zwracanym przez Lambdę.

Lokalna aplikacja kliencka składa się z formularza HTML oraz asynchronicznego kodu JavaScript wykorzystującego interfejs 'fetch()' do komunikacji z chmurą.

4 Testowanie systemu i uzyskane wyniki

Testy potoku danych przeprowadzono etapowo. Weryfikacja pierwszego etapu zakończyła się sukcesem, potwierdzając zapis surowego pliku JSON w chmurze chociażby poprzez logi wykonania (Status: 'Succeeded', Billed Duration: 292 ms).

Uruchomienie zadania AWS Glue doprowadziło do wygenerowania poprawnych struktur kolumnowych w docelowym buckecie S3. Testy integracyjne warstwy analitycznej polegały na wykonaniu zapytania testowego w edytorze Athena:

```
1 SELECT * FROM curated_weather;
```

Zapytanie zwróciło przetworzone rekordy dla miasta Gdańsk, potwierdzając poprawne działanie transformacji Sparka.

Finałowy test interfejsu użytkownika polegał na przesłaniu pytania: *"Jaka jest temperatura w Gdańsku?"* poprzez przygotowaną stronę www. System poprawnie zmapował frazę na zapytanie SQL, odpytał dane, dokonał konwersji jednostek i wyświetlił wynik końcowy: **„Obecna temperatura w Gdańsku wynosi 26.4°C.”** z kodem odpowiedzi HTTP 200.

5 Podsumowanie i wnioski

Zaprojektowany i wdrożony system w pełni realizuje założenia projektowe. Połączenie podejścia serverless z technologiami Big Data (Apache Spark, Parquet) pozwoliło stworzyć architekturę wysoce wydajną, bezpieczną i optymalną kosztowo (opłaty naliczane są wyłącznie za faktyczny czas procesowania zapytań i przestrzeń dyskową S3).

Zastosowanie mapowania opartego na szablonach w warstwie asystenta języka naturalnego sprawdziło się idealnie dla zdefiniowanego zakresu pytań. W ramach dalszego rozwoju projektu, system można rozbudować o integrację z usługą Amazon Bedrock w celu obsługi bardziej złożonych i zróżnicowanych lingwistycznie zapytań użytkowników przy użyciu dużych modeli językowych (LLM).